Name: Puneet Lingashettar
Sec: G
SRN: PES2UG24CS382

Phase 1:

```
puneet@MSI:~/PES2UG24CS382-pes-vcs$ make clean
make
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o test_tree.o
rm -rf .pes
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
```

```
puneet@MSI:~/PES2UG24CS382-pes-vcs$ # Build and run the object store test program
make test_objects
./test_objects
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -o test_objects test_objects.o object.o -lcrypto
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

```
puneet@MSI:~/PES2UG24CS382-pes-vcs$ find .pes/objects -type f | sort
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
puneet@MSI:~/PES2UG24CS382-pes-vcs$
```

Phase2:

```
puneet@MSI:~/PES2UG24CS382-pes-vcs$ make test_tree
make: 'test_tree' is up to date.
puneet@MSI:~/PES2UG24CS382-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

```
puneet@MSI:~/PES2UG24CS382-pes-vcs$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
puneet@MSI:~/PES2UG24CS382-pes-vcs$ xxd .pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf | head -20
00000000: 626c 6f62 2031 3500 5465 7374 2069 6e74  blob 15.Test int
00000010: 6567 7269 5879 0a                        egriXy.
puneet@MSI:~/PES2UG24CS382-pes-vcs$
```

Phase3

```
puneet@MSI:~/PES2UG24CS382-pes-vcs$ # Clean and rebuild everything
make clean
make pes

# Initialize a fresh repository
rm -rf .pes
./pes init

# Create test files
echo "hello" > file1.txt
echo "world" > file2.txt

# Stage the files
./pes add file1.txt file2.txt

# Show staging status
./pes status

# Display the raw index file content (for screenshot 3B)
cat .pes/index

# (Optional) Verify blob objects were created
find .pes/objects -type f
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o test_tree.o
rm -rf .pes
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  a.txt
  untracked:  commit.h
  untracked:  Makefile
  untracked:  .gitignore
  untracked:  tree.c
  untracked:  README.md
  untracked:  object.c
  untracked:  test_objects.c
  untracked:  pes.c
  untracked:  pes.h
  untracked:  test_tree.c
  untracked:  test_sequence.sh
  untracked:  commit.c
  untracked:  tree.h
  untracked:  index.c
  untracked:  index.h

100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776761501 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776761501 6 file2.txt
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
puneet@MSI:~/PES2UG24CS382-pes-vcs$
```

Phase4:

```
puneet@MSI:~/PES2UG24CS382-pes-vcs$ rm -rf .pes
make clean
make pes
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o test_tree.o
rm -rf .pes
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
puneet@MSI:~/PES2UG24CS382-pes-vcs$ ./pes init
echo "Hello" > hello.txt
./pes add hello.txt
./pes commit -m "Initial commit"
Initialized empty PES repository in .pes/
Committed: 3e06e22e705c... Initial commit
puneet@MSI:~/PES2UG24CS382-pes-vcs$ echo "World" >> hello.txt
./pes add hello.txt
./pes commit -m "Add world"
Committed: 4cd09c76b5fb... Add world
puneet@MSI:~/PES2UG24CS382-pes-vcs$ echo "Goodbye" > bye.txt
./pes add bye.txt
./pes commit -m "Add farewell"
Committed: bd3e30ef1fc4... Add farewell
puneet@MSI:~/PES2UG24CS382-pes-vcs$ ./pes log
commit bd3e30ef1fc48e049f989ff7c354f914761321908f34bbd13bf4fa62892c1b65
Author: PES User <pes@localhost>
Date:   1776761606

    Add farewell

commit 4cd09c76b5fbf4852ff80c3a43f18e80155f15477d2dbfd4b478877202480144
Author: PES User <pes@localhost>
Date:   1776761596

    Add worldell

commit 3e06e22e705cf599ffa95e71648c9d6551a354409082e15431be09c204cea0ec
Author: PES User <pes@localhost>
Date:   1776761585

    Initial committial commiQ

puneet@MSI:~/PES2UG24CS382-pes-vcs$ cat .pes/refs/heads/main
cat .pes/HEAD
bd3e30ef1fc48e049f989ff7c354f914761321908f34bbd13bf4fa62892c1b65
ref: refs/heads/main
puneet@MSI:~/PES2UG24CS382-pes-vcs$
```

Phase 5 and 6:

Branching and Checkout

Q5.1: A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout <branch> — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

A5.1:

A branch is a file under `.pes/refs/heads/` that stores a commit hash. To implement `pes checkout <branch>`:

1. Validate that `.pes/refs/heads/<branch>` exists.
2. Read target commit hash from that branch file.
3. Resolve target commit -> root tree from object store.
4. Compare current working tree state vs index/HEAD for safety (dirty-check).
5. If safe, rewrite working directory to match target tree:
   - Create missing files/directories from target tree blobs/trees.
   - Remove tracked files/directories that do not exist in target tree.
   - Apply file modes (100644/100755).
6. Rewrite index to match checked-out tree snapshot metadata.
7. Update `.pes/HEAD` to `ref: refs/heads/<branch>`.

Why this is complex:

- Requires full tree materialization from object store into filesystem.
- Must detect and prevent data loss on local edits.
- Needs careful handling of deletes, nested directories, and file mode changes.
- Must keep HEAD, index, and working directory mutually consistent.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

A5.2:

Dirty conflict detection (using index + object store):

1. For each tracked file in index:
   - `stat()` working file.
   - If missing -> unstaged deletion.
   - If mtime/size changed, re-hash file as blob and compare with index hash.
   - If hash differs, file is dirty.
2. Resolve current HEAD tree and target branch tree.
3. For each dirty tracked file, check whether the file differs between current HEAD tree and target tree:

- If target blob hash != current HEAD blob hash (or file added/removed in target), switching would overwrite semantic content.
4. If any such path exists, abort checkout and print conflicting paths.

This avoids false positives from timestamp-only changes and catches real overwrite risks.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

A5.3:
Detached HEAD means `.pes/HEAD` stores a commit hash directly (not `ref: ...`). New commits in this state:

- Still get created normally with parent links.
- Advance HEAD to newer detached commits.
- Are not referenced by any branch name.

Risk: user can "lose" them after moving HEAD elsewhere, because no branch keeps them reachable.

Recovery:
- If user still knows commit hash (or from reflog-like history if implemented), create a new branch file under `.pes/refs/heads/<name>` pointing to that commit.
- Then switch HEAD to that branch (`ref: refs/heads/<name>`).

Garbage Collection and Space Reclamation
Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

A6.1:
Safe mark-and-sweep algorithm:

1. Build root set = all refs under `.pes/refs/heads/*` (and tags if present), plus detached HEAD hash if HEAD is direct.
2. Mark phase (graph traversal):
   - Use DFS/BFS from each root commit.
   - For each commit: mark commit object, then visit its tree and parent commit.
   - For each tree: mark tree object, then visit each entry hash (blob/tree).
   - Blobs are terminal nodes.
3. Sweep phase:

- Enumerate all files under `.pes/objects/*/*`.
- Delete objects not in marked set.

Efficient structure: hash set keyed by 32-byte hash (or hex string) for O(1)-average membership checks.

Scale estimate (100,000 commits, 50 branches):
- Traversal visits all reachable commits at least once, but shared history prevents multiplicative blow-up.
- Roughly O(commits + trees + blobs) reachable objects; commonly a few hundred thousand to a few million objects depending on churn and repository size.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?
A6.2:
Concurrent GC vs commit race:

- Commit process may write new tree/blob/commit objects, then update branch ref at the end.
- If GC snapshots refs before branch update, new objects are not yet reachable from any ref.
- GC could sweep those just-written objects.
- Commit then updates ref to point to a commit whose subtree objects were deleted -> broken history.

How real Git avoids this (conceptually):
- Uses reachability protections and grace windows (recent objects are preserved).
- Uses lockfiles/coordination around ref updates and maintenance.
- Packs/prunes with safety constraints so objects concurrently being written are not immediately pruned.
- Reflogs and conservative prune horizons reduce accidental loss.